



TAYLOR'S UNIVERSITY

Wisdom • Integrity • Excellence

SCHOOL OF COMPUTER SCIENCE

Master of Applied Computing (MAC)

Assignment 1 (Weightage 30%)

December 2025 Semester

MODULE NAME : Principles of Data Science

MODULE CODE : ITS 70404

DUE DATE/TIME : 31st Dec 2025 / 23:59

PLATFORM : MyTIMeS

This paper consists of FOUR (4) pages, inclusive of this page.

Student Name

Naufal Khana Alifian

Score out of 15

Student ID

0351739

Module Learning Outcome (MLO)

MLO 2: Analyze the tools to build data-science applications for different types of data to solve real world problem.

Assessment Criteria

Assessment Task	Weightage	MLO Assessed	Formative/ Summative	Assessment Instrument	Topics	Week	MCQ2.0
Assignment 1	30%	MLO 2	Formative	Individual Assignment	3,4	4	C1, C2, C3, C5

C1 = Knowledge & Understanding; C2 = Cognitive Skills; C3A = Practical Skills; C3B = Interpersonal Skills; C3C = Communication Skills; C3D = Digital Skills; C3E = Numeracy Skills; C3F = Leadership, Autonomy & Responsibility; C4A = Personal Skills; C4B = Entrepreneurial Skills; C5 = Ethics & Professionalism.

1. Understanding the Variables (6 marks)

Identify and clearly describe each column in the EV Market & Orders dataset

1. *order_id* (string): a unique identifier for the orders and used to track the order transaction.
2. *customer_id* (string): provides unique identifier for each customer to track their transactions.
3. *order_date* (datetime): The date an order was placed on.
4. *delivery_date* (datetime): The date the product was delivered
5. *customer_segment* (categorical, nominal): classifies the types of customers into segments of either 'personal', corporate, 'fleet' or 'government' use.
6. *region* (categorical, nominal): shows the region in Southeast Asia where the sale is made.
7. *country* (categorical, nominal): shows which country the order was placed.
8. *city* (categorical, nominal): shows which city the order was placed.
9. *ev_model* (categorical, nominal): shows what type of vehicle model was purchased
10. *battery_capacity_kwh* (numerical, float): Shows the battery capacity in kilowatts per hour.
11. *range_km* (numerical, float): Shows the total range the vehicle can cover on a single full charge.
12. *charging_type* (categorical, nominal): shows what type of charging is used for the vehicle.
13. *autopilot_package* (Boolean): indicates whether it includes autopilot driving or not.
14. *purchase_channel* (categorical, nominal): shows how the vehicles was purchased and through what.
15. *base_price_usd* (numerical, float): shows the standard price, in USD, of the vehicle before any additional fees or discounts are applied.
16. *incentives_usd* (numerical, float): shows the amount, in USD, of discounts or added payment to the vehicle.
17. *final_price_usd* (numerical, float): shows the final price, in USD, needed to pay for the vehicle after the incentives used.
18. *warranty_years* (numerical, float): indicates the amount in years, how long the warranty of the vehicle is.

19. *service_plan* (categorical, ordinal): indicates the coverage plan of the vehicles with 'standard', 'extended' and 'premium'.
20. *customer_age* (numerical, Int): shows the age of the customers.
21. *satisfaction_score* (numerical, float): shows the satisfaction score of the customer with the product out of 10.
22. *issues_reported* (categorical, nominal): displays and reports the common type of issues related with the product, if there is one.
23. *returned* (Boolean): indicates whether the product has been returned to the sender or not, perhaps due to issues.
24. *energy_consumption_kwh_per_100km* (numerical, float): measures energy efficiency of battery after each 100km.
25. *mileage_after_1_year_km* (numerical, float): the mileage/distance the vehicle can travel in a year.
26. *maintenance_cost_year1_usd* (numerical, float): total cost, in USD, of maintain the product in a year.

Discuss potential relationships between key variables

- *battery_capacity_kwh* & *range_km*: perhaps larger batteries can go for longer ranges for vehicles.
- *service_plan* & *maintenance_cost_year1_usd*: maybe better service plans have lower costs for maintenance.
- *incentives_usd* & *final_price_usd*: perhaps better incentives and discounts lead to a lower final purchase price.
- *battery_capacity_kwh* & *final_price_usd*:
- *energy_consumption_kwh_per_100km* & *range_km*: perhaps some vehicles can have longer ranges with less energy consumption.
- *customer_age* & *ev_model*: perhaps different EV models are more popular based on the age range of customer.
- *country* & *final_price_usd*: different countries may have different prices on the products maybe due to relationships like taxes

2. Data Cleaning (6 marks)

Correcting Inconsistencies:

to stay consistent with all the data types, I made sure to align all data to the left and put them in lower case letters using the `.strip()` and `.lower()` function included in this code.

```
obj_cols = df.select_dtypes(include='object').columns
for col in obj_cols:
    if df[col].dtype == 'object':
        df[col] = df[col].str.strip().str.lower()

print("Cleaned DataFrame:")
df.head()
```

```
Cleaned DataFrame:
  order_id  customer_id  order_date  delivery_date  customer_segment  region  country  city  ev_model  battery_capacity_kwh  ...  final_price_usd  warranty_years  service_plan  customer_age
0    500000         2126   11-04-2023   22-nov-2023         personal    east  thailand  bangkok  ecoride 5          75.0  ...      49930.0           5      standard          61.0
1    500001         2459   2023-12-15   12-19-2023         personal    south  malaysia  penang  ecoride 3          NaN  ...      48070.0           5      premium          NaN
2    500002         1860   07-24-2024   08-13-2024         personal    east  singapore  singapore  ecoride 3          60.0  ...      42730.0           5      standard          36.0
3    500003         2294   2022-08-30   09-18-2022         corporate    east  thailand  NaN  voltx s          90.0  ...      27600.0           5      extended          65.0
4    500004         2130   02-jan-2022   2022-01-11          fleet    south  NaN  singapore  ecoride 5          60.0  ...      42830.0           5      standard          64.0
```

Handling Missing Values:

I first checked out the variables that had null data in them and the amount of null data using the code “`df.isnull().sum()`” to get this output:

```
[139] df.isnull().sum()
✓ 0s # check the sum of and null data
```

	0
order_id	0
customer_id	0
order_date	0
delivery_date	0
customer_segment	68
region	71
country	73
city	76
ev_model	0
battery_capacity_kwh	146
range_km	136

For the values that are Objects and categorical which are ‘*customer_segment*’, ‘*region*’, ‘*charging_type*’, ‘*service_plan*’ and ‘*issues_reported*’, I took the mode from each column by using this code: “`df[‘insert column name’].value_counts()`”, and then impute it into the null values using this code using:

```
df[‘insert column name’] = df[‘insert column name’].fillna(‘ insert the mode value’)
```

Example for *customer_segment*:

```
[140]
✓ Os
df['customer_segment'].unique()
df['customer_segment'].value_counts()

customer_segment
personal      1439
corporate      605
fleet          344
government     104

dtype: int64
```

The sum of null data is then checked again in each column again to look out for the change. This step is then repeat this step for other categorical data types.

For the values that are numerical such as are 'battery_capacity_kwh', 'range_km', 'base_price_usd', 'final_price_usd', 'satisfaction_score', 'energy_consumption_kwh_per_100km', 'maintainence_cost_year1_usd' and 'customer_age', I took the mean from each column by using this code: ".mean()" and using the "round()" function to round up the variables. I would then use "df['insert column name'].replace(np.nan, 'insert mean value', inplace=True)" to impute the missing values.

Example for 'base_price_usd':

```
[125]
mean = df['base_price_usd'].mean()
print(round(mean,0))

... 45097.0

[128]
✓ Os
df['base_price_usd'].replace(np.nan, 45105.0, inplace=True)

/tmp/ipython-input-3881543502.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation

df['base_price_usd'].replace(np.nan, 45105.0, inplace=True)
```

For the location on 'county' and 'city', I first observed the unique values and mode amount of countries and cities first using the .value_counts() function to see the list of countries and cities. I then create a dictionary that will map the countries to cities and another for cities to countries to fill them in. I used this code to do so.

```

city_to_country = {'singapore':'singapore',
'penang':'malaysia',
'chiang mai':'thailand',
'johor bahrui':'malaysia',
'kuala lumpur':'malaysia',
'bangkok':'thailand',
'shah alam':'malaysia',
'surabaya':'indonesia',
'jakarta':'indonesia',
'ho chi minh city':'vietnam',
'hanoi':'vietnam'}

df["country"] = df["country"].fillna(df["city"].map(city_to_country))

df.head()

```

order_id	customer_id	order_date	delivery_date	customer_segment	region	country	city	ev_model	
0	500000	2126	11-04-2023	22-nov-2023	personal	east	thailand	bangkok	ecoride 5
1	500001	2459	2023-12-15	12-19-2023	personal	south	malaysia	penang	ecoride 3
2	500002	1860	07-24-2024	08-13-2024	personal	east	singapore	singapore	ecoride 3
3	500003	2294	2022-08-30	09-18-2022	corporate	east	thailand	NaN	voltx s
4	500004	2130	02-jan-2022	2022-01-11	fleet	south	singapore	singapore	ecoride 5

5 rows × 26 columns

```

country_to_city = {'singapore':'singapore',
'malaysia':'kuala lumpur',
'thailand':'bangkok',
'indonesia':'jakarta',
'vietnam':'ho chi minh city'}

df["city"] = df["city"].fillna(df["country"].map(country_to_city))

df.head(5)

```

order_id	customer_id	order_date	delivery_date	customer_segment	region	country	city	ev_model	battery_capacity_kwh	
0	500000	2126	11-04-2023	22-nov-2023	personal	east	thailand	bangkok	ecoride 5	75.0
1	500001	2459	2023-12-15	12-19-2023	personal	south	malaysia	penang	ecoride 3	NaN
2	500002	1860	07-24-2024	08-13-2024	personal	east	singapore	singapore	ecoride 3	60.0
3	500003	2294	2022-08-30	09-18-2022	corporate	east	thailand	bangkok	voltx s	90.0
4	500004	2130	02-jan-2022	2022-01-11	fleet	south	singapore	singapore	ecoride 5	60.0

5 rows × 26 columns

Removing Duplicates:

For the 'order_id' section we first check and count how many duplicate data there is and then count how many rows of the next batch of duplicates are. We use this code to check the number of duplicate values in the 'order_id' with all of them have 2 of the same (Fig 1.). We then check the initial rows of these duplicated values with this second code (Fig. 2).

```
df['order_id'].value_counts()[df['order_id'].value_counts() > 1]
```

order_id	count
500388	2
500842	2
500706	2
500066	2
502286	2
502361	2
502360	2

Fig 1.

```
df[df.duplicated(subset='order_id', keep='first')].sort_values('order_id')
```

order_id	customer_id	order_date	delivery_date	customer_segment	region	country	city	ev_model	
2546	500000	2126	2023-04-11	2023-11-22	personal	east	thailand	bangkok	ecoride 5
2517	500066	1600	2024-05-11	2024-05-30	corporate	west	malaysia	kuala lumpur	ecoride 5
2515	500149	1288	2024-12-18	2024-12-23	personal	north	malaysia	shah alam	ecoride 3
2510	500191	2881	2023-05-12	2023-12-13	personal	south	thailand	chiang mai	urbanev
2543	500204	2757	2024-04-01	2024-04-17	corporate	north	malaysia	penang	voltx s
2553	500288	1001	2023-11-13	2023-12-03	corporate	south	vietnam	hanoi	voltx s
2521	500374	1452	2022-12-14	2022-12-27	personal	west	indonesia	surabaya	ecoride 3

Fig 2.

3. Data Transformation (6 marks)

Encoding Categorical Variables:

Label Encoding:

3 variables were used of Label Encoding which were '*autopilot_package*', '*returned*' and '*service_plan*'.

Justification for '*autopilot_package*' and '*returned*': since they were only a yes or no output which is Boolean, they were represented as 0 and 1 and no and yes respectively.

Justification for '*service_plan*': Since *service_plan* is an ordinal data, I decided to represent them with label encoding since it goes in order from *standard* to *premium*.

Example code used for '*autopilot_package*':

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df['autopilot_package'] = encoder.fit_transform(df['autopilot_package'])
print(df['autopilot_package'])
print("the new types of values included are", df['autopilot_package'].unique())
```

0	0
1	0
2	0
3	1
4	1
...	...
2495	1
2496	0
2497	1
2498	0
2499	1

Name: autopilot_package, Length: 2500, dtype: int64
the new types of values included are [0 1]

One Hot Encoding

8 variables used One Hot Encoding which were *customer_segment*, *region*, *country*, *city*, *ev_model*, *charging_type*, *purchase_channel*, *issues_reported*. Since all of these 8 variables are nominal categorical therefore have no order in them therefore, One Hot Encoding is used on them to be better read in EDA.

Example of code used in for all 8 variables:


```
df = pd.get_dummies(df,
columns=['country','customer_segment','region','city','ev_model', 'charging_type', 'purchase_channel', 'issues_reported'], dtype=int,
drop_first=False
)
```

```
df.head(12)
```

	order_id	customer_id	order_date	delivery_date	battery_capacity_kwh	range_km	autopilot_package	base_price_usd	incentives_usd	final_price_usd
0	500000	2126	2023-11-04	2023-11-22	75.0	422.0	0	50830.0	0	49930.0
1	500001	2459	2023-12-15	2023-12-19	75.0	585.0	0	48450.0	0	48070.0
2	500002	1860	2024-07-24	2024-08-13	60.0	370.0	0	43590.0	1000	42730.0
3	500003	2294	2022-06-30	2022-09-18	90.0	514.0	1	32070.0	2500	27600.0
4	500004	2130	2022-02-01	2022-01-11	60.0	370.0	1	45580.0	2500	42830.0
5	500005	2095	2022-08-13	2022-08-28	75.0	447.0	1	29820.0	2500	26630.0
6	500006	2724	2024-11-03	2024-03-31	100.0	593.0	1	48970.0	0	43293.0
7	500007	2044	2023-08-12	2023-08-28	75.0	357.0	0	38150.0	5000	33400.0
8	500008	2638	2022-04-26	2022-05-16	70.0	440.0	0	43880.0	1000	44150.0
9	500009	1121	2024-07-06	2024-07-21	50.0	293.0	1	41420.0	0	39410.0
10	500010	1466	2022-07-13	2022-07-26	80.0	444.0	0	48960.0	2500	45840.0
11	500011	2238	2024-05-20	2024-06-07	100.0	603.0	1	39470.0	0	39670.0

12 rows × 11 columns

Feature Engineering:

- price per km of range: calculates the *'base_price_usd'* divided by the *'range_km'*
 - Reason: this will give a standardize measurement on the vehicle price by its range allowing a comparison of efficiency throughout the car models.
- Total revenue per order: calculates the *'final_price_usd'* with the *'maintenennce_cost_year1_usd'*.
 - Reason: this allows us to see the total amount of sales for each vehicle to get a better understanding of the full price needed for this product.
- discount percentage: gives out the percentage amount by taking the *'final_price_usd'* and dividing it with the *'base_price_usd'* to them multiply it by 100 then subtract it with 100 again to see the percentage of discount.
 - Reason: allows us to measure the discount applied to each purchase to better observe the discount strategies on how it would effect customers.
- order month from *order_date*: finds the numerical month date the order took place in.
 - Reason: allows the viewing of trends and patterns to become more simpler and better for analysis.

4. Data Analysis & Exploration (6 marks)

Descriptive Statistics:

- Average total revenue by manufactured model:
 - **Reason:** To compare and identify models that have higher value in it sales.
 - **Result:** Most bought model was the ‘Voltx Pro’ Ev suggesting more marketing and advertising could be done on this model.

```
ev_columns = ['ev_model_ecoride s', 'ev_model_familyev xl', 'ev_model_urbanev', 'ev_model_voltx pro', 'ev_model_voltx s']  
  
df['ev_model'] = df[ev_columns].idxmax(axis=1) # remeber that idxmax gives the column with the highest val and axis=1 means y line  
#used for one hot encoding remeber  
  
final_avg_price_per_model = (df.groupby(['ev_model'])['total_revenue_per_order'].mean().sort_values(ascending=True))  
  
print(round(final_avg_price_per_model,2))
```

ev_model	
ev_model_familyev xl	42975.12
ev_model_voltx s	43273.73
ev_model_ecoride 5	43750.59
ev_model_urbanev	44491.69
ev_model_voltx pro	44555.12
Name: total_revenue_per_order, dtype: float64	

- Total revenue by manufactured model:
 - **Reason:** To identify which models can bring in the most sales in total
 - **Result:** the EV model with the highest total revenue was the ‘Ecoride 5’ with about 34,956,720 USD made. This suggests that the ‘ecoride 5’ may be the most expensive due to its features and perks that are contained with it.

```
final_tot_price_per_model = (df.groupby(['ev_model'])['total_revenue_per_order'].sum().sort_values(ascending=True))  
  
print(final_tot_price_per_model)
```

ev_model	
ev_model_voltx pro	18356709.4
ev_model_urbanev	18508541.6
ev_model_familyev xl	18565251.9
ev_model_voltx s	19083712.8
ev_model_ecoride 5	34956719.8
Name: total_revenue_per_order, dtype: float64	

- Average total revenue by region:
 - **Reason:** To identify the sales and purchase through the regions to see where customers tend to spend more in.

- **Result:** The region with the majority of its sales being done is in the south. This can suggest that many buyers of the EV are located more in the south side which more marketing could be pushed on those regions.

```
regions = ['region_north', 'region_south', 'region_east', 'region_west']

df['country_to_fin_price'] = df[regions].idxmax(axis=1) # remember that idxmax gives the column with the highest val and axis=1 means y line
#used for one hot encoding remember

avg_price_by_country = (df.groupby(['country_to_fin_price'])['total_revenue_per_order'].mean().sort_values(ascending=False))

print(round(avg_price_by_country,2))
```

country_to_fin_price	total_revenue_per_order
region_south	44058.24
region_east	43914.79
region_west	43675.16
region_north	43446.87

Name: total_revenue_per_order, dtype: float64

- Average total revenue by country:
 - **Reason:** To identify the sales and purchase through the countries to understand which country may offer better revenue in sales and where more likely customers may be held in.
 - **Result:** While all countries have almost the same sales amount, Thailand has the most by 300 more meaning that the company could spend more marketing in Thailand or boost Marketing in other countries than Thailand.

```
country_columns = ['country_malaysia', 'country_singapore', 'country_thailand', 'country_vietnam', 'country_indonesia']

df['country_to_fin_price'] = df[country_columns].idxmax(axis=1) # remember that idxmax gives the column with the highest val and axis=1 means y line
#used for one hot encoding remember

avg_price_by_country = (df.groupby(['country_to_fin_price'])['total_revenue_per_order'].mean().sort_values(ascending=False))

print(round(avg_price_by_country,2))
```

country_to_fin_price	total_revenue_per_order
country_thailand	44204.08
country_malaysia	43937.64
country_singapore	43708.61
country_vietnam	43333.63
country_indonesia	43252.63

Name: total_revenue_per_order, dtype: float64

- Average discount percentage by model:
 - **Reason:** To understand which models rely more on discounts, allowing the use of better pricing and incentives to drive higher sales.
 - **Result:** highest discount at 4.63% goes to the Family EV XL model perhaps due to its high costs out of all other models with the discount proportionate to the pricing.

```
average_disc_per_model = (df.groupby(['ev_model'])['discount_percentage(%)'].mean().sort_values(ascending=True))

print(round(average_disc_per_model,2))
```

ev_model	discount_percentage(%)
ev_model_voltx pro	2.97
ev_model_urbanev	3.37
ev_model_voltx s	3.89
ev_model_ecoride 5	4.14
ev_model_familyev xl	4.63

Name: discount_percentage(%), dtype: float64

Correlation Analysis

- battery capacity vs. range: 0.48

Justification: Used to understand and verify if larger batteries equate to longer driving ranges.

```
correlation = df['battery_capacity_kwh'].corr(df['range_km'])
round(correlation, 2)

np.float64(0.48)
```

- battery capacity vs price per km: -0.73

Justification: Used to understand and verify if larger batteries are more cost efficient in terms of their range.

```
correlation = df['battery_capacity_kwh'].corr(df['price_per_km'])
round(correlation, 2)

np.float64(-0.73)
```

- vehicle price vs. performance metrics:
 - range_km: -0.01
 - energy_consumption_kwh_per_100km: 0.05
 - 'mileage_after_1_year_km: 0.00

Justification: Comparing the performance against the price helps to determine if more expensive vehicles lead to better performance.

```
round(df[['final_price_usd', 'range_km', 'energy_consumption_kwh_per_100km', 'mileage_after_1_year_km']].corr(), 2)
```

	final_price_usd	range_km	energy_consumption_kwh_per_100km	mileage_after_1_year_km
final_price_usd	1.00	-0.01	0.05	-0.00
range_km	-0.01	1.00	-0.01	-0.01
energy_consumption_kwh_per_100km	0.05	-0.01	1.00	-0.01
mileage_after_1_year_km	-0.00	-0.01	-0.01	1.00

- discount vs. total revenue: -0.73

Justification: Understanding discounts on how they affect total revenue help understand how effective the strategies are.

```
correlation = df['discount_percentage(%)'].corr(df['total_revenue_per_order'])
round(correlation, 2)

np.float64(-0.73)
```

- Base price vs final price: 0.7

Justification: used to understand how the relationship between the two variables confirms how much is being paid by the customer.

```
corralation = df['base_price_usd'].corr(df['final_price_usd'])
round(corralation, 2)

np.float64(0.7)
```

- Final price vs discount percentage (%): -0.73

Justification: showed to understand how much the discounts can affect the final amount of the product.

```
corralation = df['final_price_usd'].corr(df['discount_percentage(%)'])
round(corralation, 2)

np.float64(-0.73)
```

5. Conclusion & References (6 marks)

Summarize main key analytical findings

- **battery capacity vs. range: 0.48**

Analysis: this shows a moderate positive relationship meaning that vehicles with larger batteries do have a longer drive range, the relationship correlation is not perfect. There may be a variety of other factors that influence the range that may not be found in this dataset.

- **battery capacity vs price per km: -0.73**

Analysis: there is a strong negative correlation which shows that the larger the battery capacity is, the cost per km decreases. This shows a strong correlation that vehicles with a larger battery capacity are more efficient in cost for the price per km, showing that the higher price is can be worth the efficiency.

- **vehicle price vs. performance metrics:**

- range_km: -0.01
- energy_consumption_kwh_per_100km: 0.05
- 'mileage_after_1_year_km: 0.00

Analysis: Due to all correlations leading to almost 0, there is no concrete relationship between the vehicle pricing and its performance metrics. This can say that a higher price for vehicles does not mean it will have a better range, energy consumption and mileage. This also shows that price may be determined by other factors such as EV model.

- **discount vs. total revenue: -0.73**

Analysis: Another strong negative correlation shows that higher discounts and incentives strongly equate to lower revenue. This is a common trend in all discounted products.

- **base_price vs final price: 0.7**

Analysis: this strong positive correlation shows that the final price is often related and follows that base price meaning that even with the usage of discounts and incentives. The final price would still be slightly higher than the base price following this relationship.

- **final_price_usd vs discount percentage (%): -0.73**

Analysis: A strong negative correlation shows that a higher discount percentage does signify a reduce in the final price meaning that discounts do have a large impact on the final amounts customers pay.

- **Price_per_km to base_price_usd: 0.55**

Analysis: This shows a moderate positive relationship showing that when the base price increases, the cost per km may increase as well however, it may also decrease or have no effect on the cost per km. This shows that base price of vehicles do not always translate to lower or higher cost per km. other factors may also play other roles or even have more effect on the base price itself.

Provide business-oriented insights and recommendations

- Since battery capacity has a moderate positive relationship towards range, manufacturing companies for these EV cars should instead work to create better efficient and optimized batteries rather than just increasing battery size alone.
- With the knowledge that larger battery capacity gives a better price per km, it is best to advertise, price and market these EV cars more to more cost-oriented customers.
- As we can tell that vehicle price has nothing to do with performance, companies should instead justify their higher price tags on vehicles based on other factors like services, features and perks rather than relying everything on vehicle performance.
- With discounting being quite relevant in the relationship to the total revenue, companies can insist on creating more discounted offers, season discounts and incentives to drive up more sales.
- With base price affecting strong to the final price of the vehicle, carefully ensure that the base price is fair and even in terms of what product is being sold while also trying to create the best possible price out of it to gain maximum profit, as the final price can be strongly affected by these factors and it comes down to the customer who is willing to pay for that price.

Included References

- <https://www.w3schools.com/>
- <https://www.geeksforgeeks.org/data-science/data-science-fundamentals/>
- Dr. Obay's Data Wrangling notes/exercises 29/11/2025 – 20/12/2025
- Google Colab link:
https://colab.research.google.com/drive/1GroDrSzkHU9kSpR55b_ZTik2r7B0oHTl?usp=sharing